Project

On

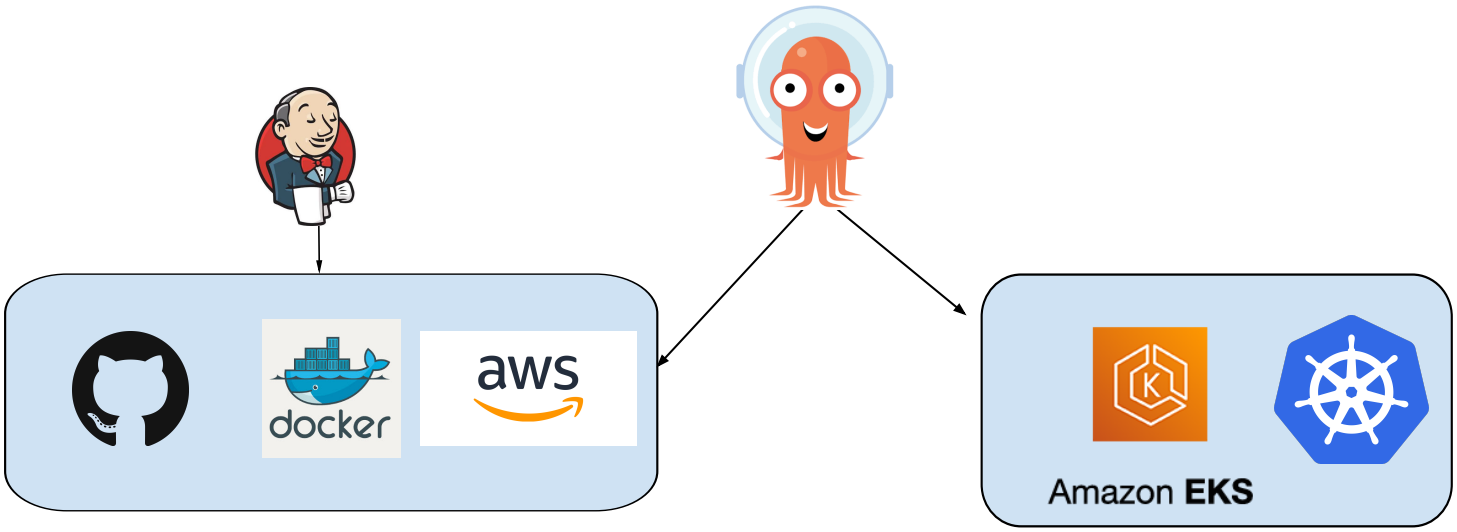
End-to-End Automated 3 Tier-Application

using Jenkins & ArgoCD

By :

Shivam Kumar Singh

B.Tech in Computer Science & Engineering (2026)



Cloud-Native 3-Tier E-Commerce Pipeline: End-to-End Automation with AWS, Jenkins, EKS, and Argo CD

Executive Project Overview

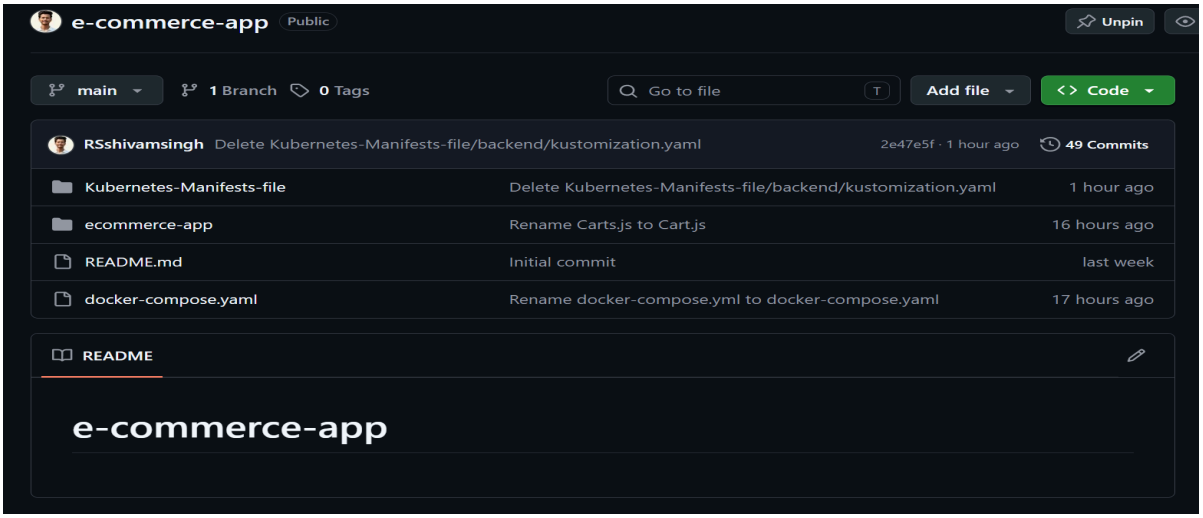
This project demonstrates an enterprise-grade cloud-native deployment and automated lifecycle management of a microservices-based e-commerce platform. The architecture implements a two-stage CI/CD and GitOps workflow: continuous integration via **Jenkins** (handling code analysis, image building, and pushing to **Docker Hub**), paired with continuous deployment via **Argo CD** on **Amazon EKS (Elastic Kubernetes Service)** with dynamic path routing through an **AWS Application Load Balancer (ALB)** Ingress.

System Architecture & Stack Breakdown

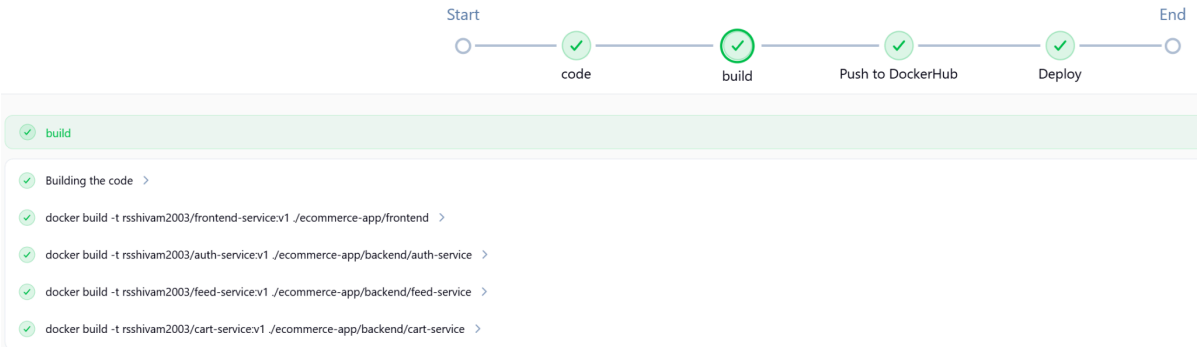
Stack 1: Continuous Integration & Infrastructure Management (DevOps Phase)

- **Version Control System (VCS)**: Git & GitHub
- **Host Compute**: AWS EC2 Instance (Ubuntu 22.04 LTS)
- **CI Orchestration Engine**: Jenkins CI/CD Pipeline
- **Containerization**: Docker, Dockerfile per microservice
- **Image Registry**: Docker Hub (`rsshivam2003/*`)
- **AWS IAM**: Created IAM user and provide permissions

Git URL: https://github.com/RSshivamsingh/e-commerce-app.git



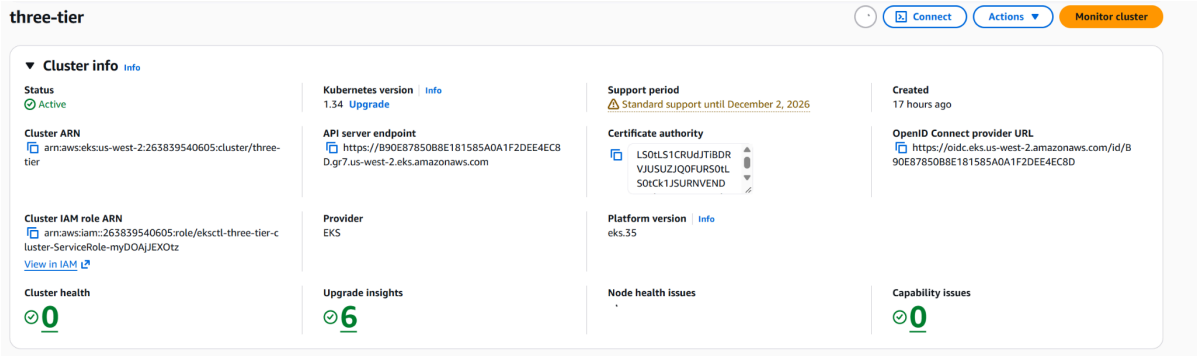
Jenkins PipeLine and stages :-



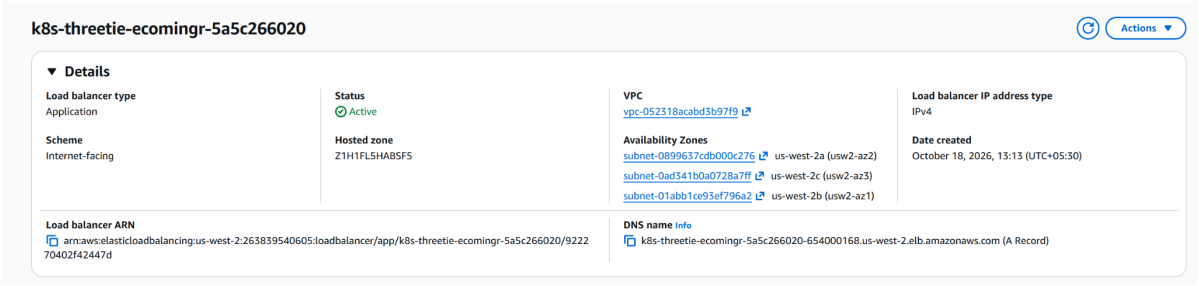
Stack 2: Container Orchestration & GitOps Deployment (Kubernetes & Argo CD Phase)

- **Kubernetes Cluster Engine**: Amazon EKS (Kubernetes v1.34)
- **Microservices Application Tier**:
  - **Frontend**: Nginx static web server (`frontend-service`)
  - **Backend API Services**: Auth (`auth-service`), Feed (`feed-service`), Cart (`cart-service`)
  - **Database**: MongoDB Stateful Deployment (`mongodb-svc`) with PVC storage & Kubernetes Secrets (`mongo-sec`)
- **Ingress & Load Balancing**: AWS Load Balancer Controller managing an Internet-Facing AWS Application Load Balancer (ALB)
- **GitOps Engine**: Argo CD (Syncing declarative Kustomize structure directly from GitHub)

Kubernetes (eks-cluster):-



AWS Load balancer:-



Section 1: Stack 1 - Build & CI/CD Pipeline Commands

1. EC2 Instance & Environment Setup

System Package Update & Docker Installation

sudo apt update && sudo apt upgrade -y

sudo apt install -y docker.io git curl

Add user to Docker group

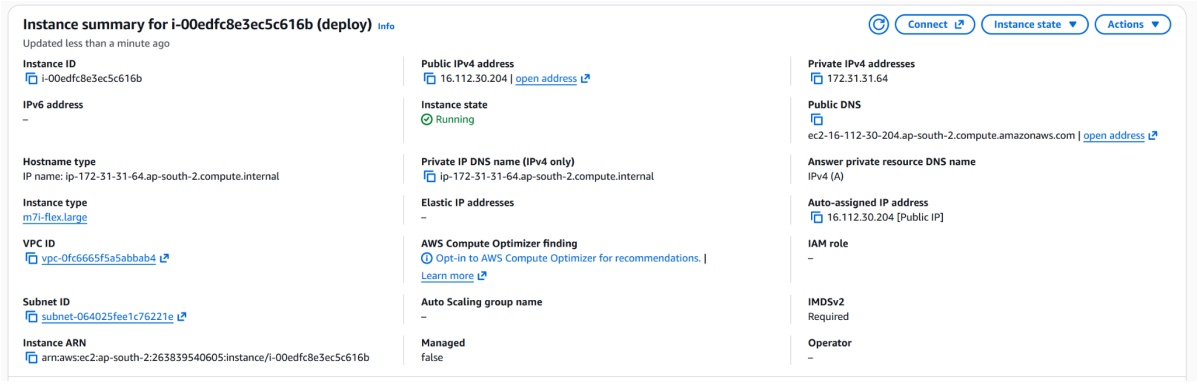
sudo usermod -aG docker $USER

newgrp docker

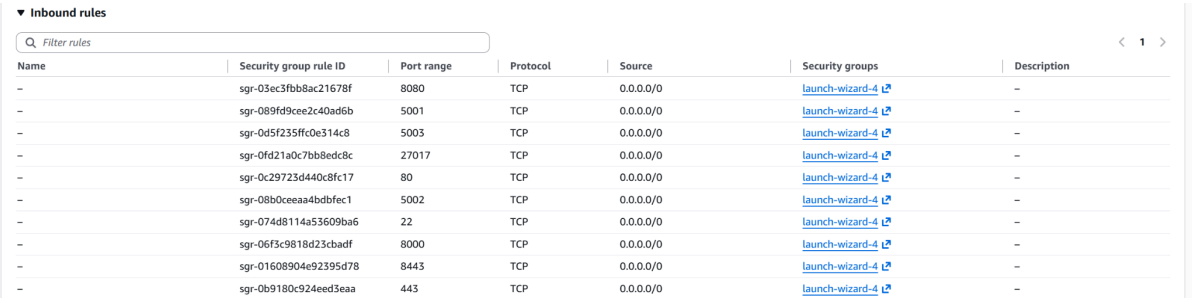
Verify Docker Installation

docker --version

AWS EC2 [redacted]



Allowed security groups:



▼ Inbound rules

Filter rules

1

Name	Security group rule ID	Port range	Protocol	Source	Security groups	Description
–	sgr-03ec3fbb8ac21678f	8080	TCP	0.0.0.0/0	launch-wizard-4	–
–	sgr-089fd9cee2c40ad6b	5001	TCP	0.0.0.0/0	launch-wizard-4	–
–	sgr-0d5f235ffc0e314c8	5003	TCP	0.0.0.0/0	launch-wizard-4	–
–	sgr-0fd21a0c7bb8edc8c	27017	TCP	0.0.0.0/0	launch-wizard-4	–
–	sgr-0c29723d440c8fc17	80	TCP	0.0.0.0/0	launch-wizard-4	–
–	sgr-08b0ceeaa4bdbfec1	5002	TCP	0.0.0.0/0	launch-wizard-4	–
–	sgr-074d8114a53609ba6	22	TCP	0.0.0.0/0	launch-wizard-4	–
–	sgr-06f3c9818d23cbadf	8000	TCP	0.0.0.0/0	launch-wizard-4	–
–	sgr-01608904e92395d78	8443	TCP	0.0.0.0/0	launch-wizard-4	–
–	sgr-0b9180c924eed3eaa	443	TCP	0.0.0.0/0	launch-wizard-4	–

2. Microservice Dockerfile Construction Examples

*(Dockerfile for Frontend)*FROM nginx:alpine

COPY . /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

*(Dockerfile for Node.js Backends - Auth, Feed, Cart)*FROM node:18-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 5001

CMD ["npm", "start"]

3. Image Build & Docker Hub Push CLI Commands

Docker Hub Login

docker login -u rsshivam2003

Build Container Images

docker build -t rsshivam2003/frontend-service:v1 ./frontend

docker build -t rsshivam2003/auth-service:v1 ./microservices/auth-service

docker build -t rsshivam2003/feed-service:v1 ./microservices/feed-service

docker build -t rsshivam2003/cart-service:v1 ./microservices/cart-service

Push Images to Registry

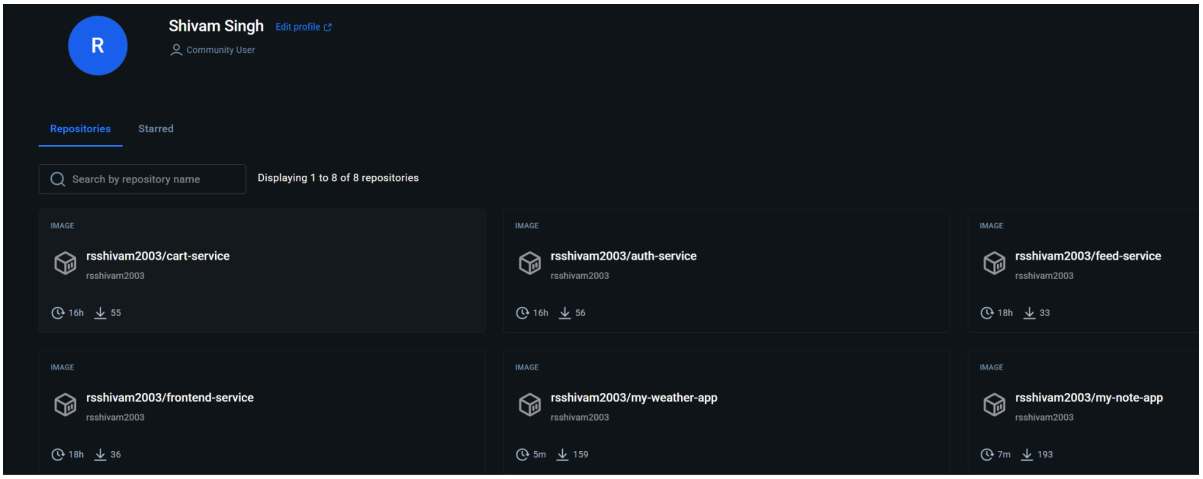
docker push rsshivam2003/frontend-service:v1

docker push rsshivam2003/auth-service:v1

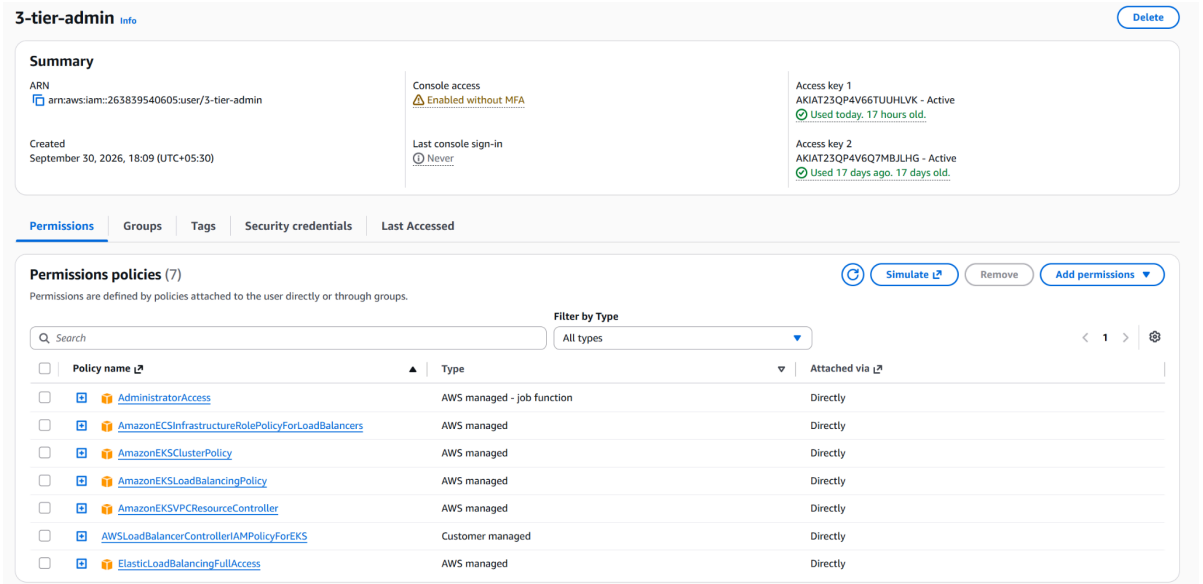
docker push rsshivam2003/feed-service:v1

docker push rsshivam2003/cart-service:v1

DockerHub Repositories:-
URL:- https://hub.docker.com/u/rsshivam2003



4. AWS IAM permissions policies



Section 2: Stack 2 - Kubernetes Manifests, Ingress & Argo CD Setup

1. Cluster & Namespace Provisioning CLI

Connect local kubectl to Amazon EKS Cluster

aws eks update-kubeconfig --region us-west-2 --name three-tier

Create Application Namespace

kubectl create namespace three-tier

2. Database Manifests (`database/`)

- **Deployment & PVC**: Provisioned persistent storage (`mongo-pvc`, 5Gi) and MongoDB container attached to secret credentials.
- **CLI Execution**:

kubectl apply -f database/secrets.yaml -n three-tier

kubectl apply -f database/pvc.yaml -n three-tier

kubectl apply -f database/deployment.yaml -n three-tier

kubectl apply -f database/service.yaml -n three-tier

```
ubuntu@ip-172-31-31-64:~/e-commerce-app$ kubectl get svc -n three-tier
NAME               TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)     AGE
auth-service       ClusterIP   10.100.151.0     <none>        5001/TCP    17h
cart-service       ClusterIP   10.100.222.115   <none>        5003/TCP    17h
feed-service       ClusterIP   10.100.213.74    <none>        5002/TCP    17h
frontend-service   ClusterIP   10.100.248.56    <none>        80/TCP      17h
mongodb-svc        ClusterIP   10.100.185.62    <none>        27017/TCP   17h
ubuntu@ip-172-31-31-64:~/e-commerce-app$
```

3. Backend Microservices Manifests (`backend/`)

- Contains unified multi-document deployment and service configs for `auth-service` (Port 5001), `feed-service` (Port 5002), and `cart-service` (Port 5003).
- **CLI Execution**:

kubectl apply -f backend/deployment_service.yaml -n three-tier

4. Frontend Web Manifests (`frontend/`)

- Exposes Nginx container on Port 80 via `ClusterIP` (`frontend-service`).
- **CLI Execution**:

h
kubectl apply -f frontend/deployment.yaml -n three-tier
kubectl apply -f frontend/service.yaml -n three-tier

5. Ingress Controller & AWS ALB Manifest (`ingress.yaml`)

```yaml

apiVersion: networking.k8s.io/v1

kind: Ingress

metadata:

  name: ecom-ingress

  namespace: three-tier

  annotations:

    alb.ingress.kubernetes.io/scheme: internet-facing

    alb.ingress.kubernetes.io/target-type: ip

    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}]'

spec:

  ingressClassName: alb

  rules:

    - host: amanpathakdevops.study

      http:

        paths:

          - path: /api/auth
```

```
    pathType: Prefix
    backend:
      service:
        name: auth-service
        port:
          number: 5001
  - path: /api/feed
    pathType: Prefix
    backend:
      service:
        name: feed-service
        port:
          number: 5002
  - path: /api/cart
    pathType: Prefix
    backend:
      service:
        name: cart-service
        port:
          number: 5003
  - path: /
    pathType: Prefix
    backend:
```

service:

name: frontend-service

port:

number: 80

- **CLI Execution**:

kubectl apply -f ingress.yaml -n three-tier

kubectl get ingress -n three-tier

Runnning PODS:-

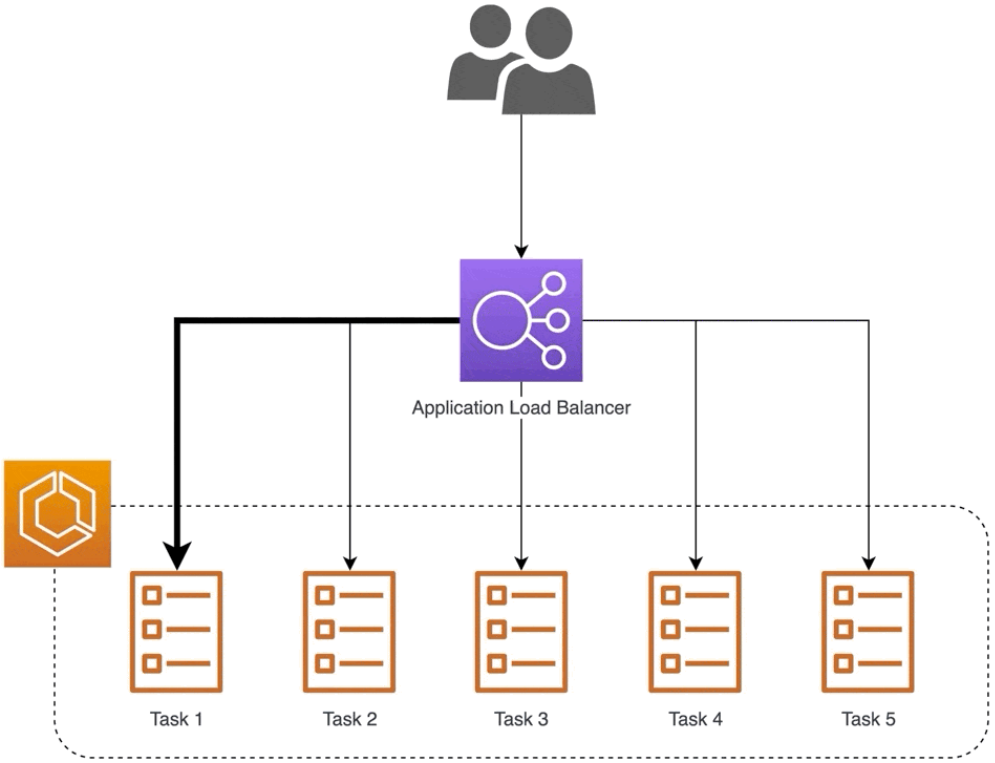
```
ubuntu@ip-172-31-31-64:~/e-commerce-app$ kubectl get pods -n three-tier
NAME                                  READY   STATUS    RESTARTS   AGE
auth-deployment-7889647f8-8vzgv       1/1     Running   0          16h
cart-deployment-7ff99d8746-2fqk2      1/1     Running   0          16h
cart-deployment-7ff99d8746-nb9ls      1/1     Running   0          16h
feed-deployment-54c4b46c97-cwx5k      1/1     Running   0          17h
feed-deployment-54c4b46c97-hh5th      1/1     Running   0          17h
frontend-deployment-d5dc5c8bb-bzvl2   1/1     Running   0          17h
frontend-deployment-d5dc5c8bb-smtkx   1/1     Running   0          17h
mongodb-569cc449d8-2qwfc              1/1     Running   0          17h
ubuntu@ip-172-31-31-64:~/e-commerce-app$
```

```
ubuntu@ip-172-31-31-64:~/e-commerce-app$ kubectl get all -n three-tier
NAME                                      READY   STATUS    RESTARTS   AGE
pod/auth-deployment-7889647f8-8vzgv       1/1     Running   0          16h
pod/cart-deployment-7ff99d8746-2fqk2      1/1     Running   0          16h
pod/cart-deployment-7ff99d8746-nb9ls      1/1     Running   0          16h
pod/feed-deployment-54c4b46c97-cwx5k      1/1     Running   0          17h
pod/feed-deployment-54c4b46c97-hh5th      1/1     Running   0          17h
pod/frontend-deployment-d5dc5c8bb-bzvl2   1/1     Running   0          17h
pod/frontend-deployment-d5dc5c8bb-smtkx   1/1     Running   0          17h
pod/mongodb-569cc449d8-2qwfc              1/1     Running   0          17h

NAME                       TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)     AGE
service/auth-service       ClusterIP   10.100.151.0     <none>        5001/TCP    17h
service/cart-service       ClusterIP   10.100.222.115   <none>        5003/TCP    17h
service/feed-service       ClusterIP   10.100.213.74    <none>        5002/TCP    17h
service/frontend-service   ClusterIP   10.100.248.56    <none>        80/TCP      17h
service/mongodb-svc        ClusterIP   10.100.185.62    <none>        27017/TCP   17h

NAME                                  READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/auth-deployment       1/1     1            1           17h
deployment.apps/cart-deployment       2/2     2            2           17h
deployment.apps/feed-deployment       2/2     2            2           17h
deployment.apps/frontend-deployment   2/2     2            2           17h
deployment.apps/mongodb               1/1     1            1           17h

NAME                                            DESIRED   CURRENT   READY   AGE
replicaset.apps/auth-deployment-6c886c4596      0         0         0       16h
replicaset.apps/auth-deployment-7889647f8       1         1         1       16h
replicaset.apps/auth-deployment-845bb7454d      0         0         0       17h
replicaset.apps/cart-deployment-5466df5cf8      0         0         0       17h
replicaset.apps/cart-deployment-6749948bf7      0         0         0       16h
replicaset.apps/cart-deployment-7ff99d8746      2         2         2       16h
replicaset.apps/feed-deployment-54c4b46c97      2         2         2       17h
replicaset.apps/frontend-deployment-d5dc5c8bb   2         2         2       17h
replicaset.apps/mongodb-569cc449d8              1         1         1       17h
ubuntu@ip-172-31-31-64:~/e-commerce-app$
```



Section 3: Kustomization Declarative Structure & GitOps Automation

To allow Argo CD to map the nested repository hierarchy (Kubernetes-Manifests-file/) into a connected visual tree, a Kustomize structure was added.

Root `Kubernetes-Manifests-file/kustomization.yaml`

apiVersion: kustomize.config.k8s.io/v1beta1

kind: Kustomization

resources:

- ingress.yaml

- backend/

- database/

- frontend/

Subfolder Kustomizations (`backend/kustomization.yaml`, `database/kustomization.yaml`, `frontend/kustomization.yaml`)

apiVersion: kustomize.config.k8s.io/v1beta1

kind: Kustomization

resources:

- deployment_service.yaml

Argo CD Deployment & Exposure CLI

Install Argo CD into EKS

kubectl create namespace argocd

kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

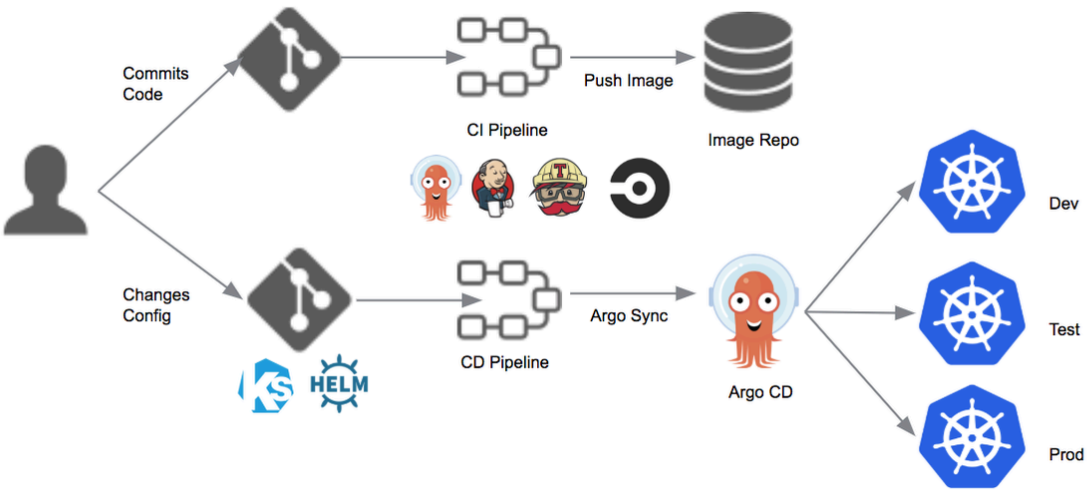
Patch Argo CD Server to LoadBalancer for direct dashboard access

kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "LoadBalancer"}}'

Retrieve Argo CD LoadBalancer Endpoint

kubectl get svc argocd-server -n argocd

ArgoCD :-

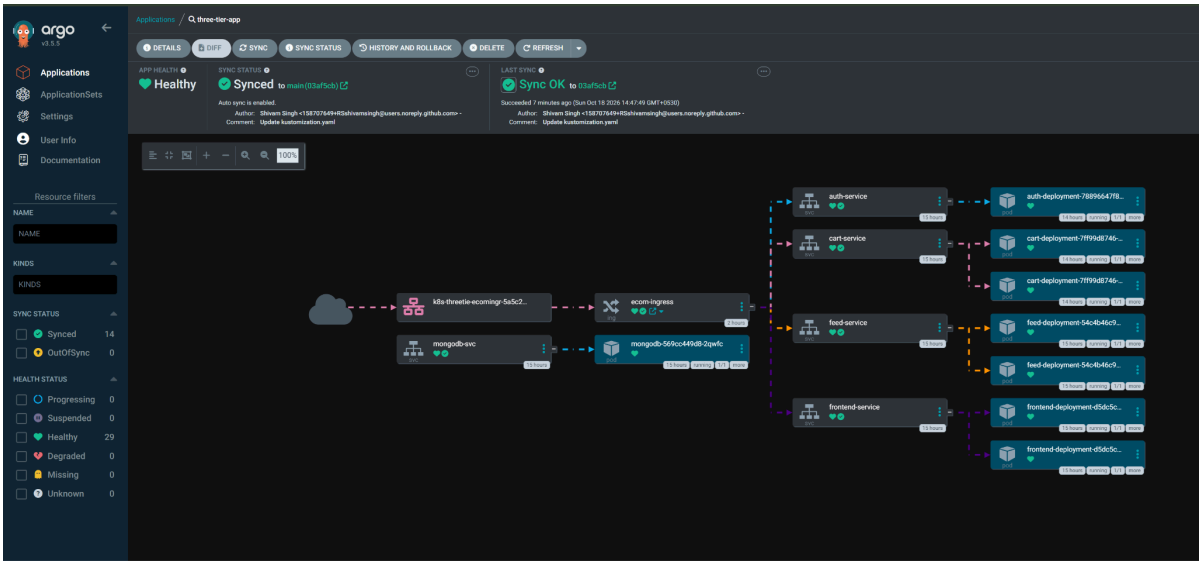


Section 4: Visual Architecture Diagram & Verification Summary

Final Argo CD Live Visual Topology Breakdown

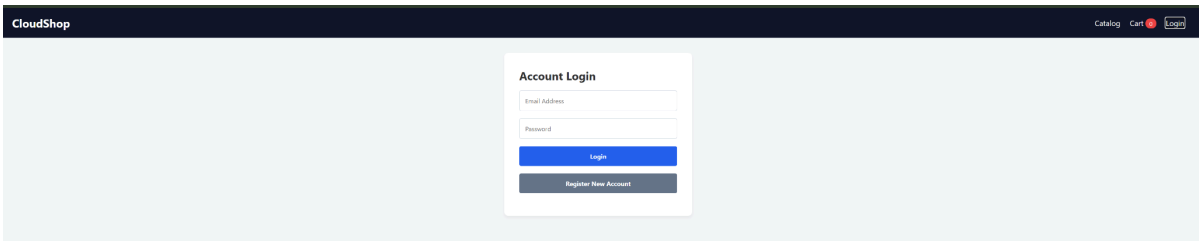
The resulting visual graph rendered inside Argo CD proves full cluster synchronization:

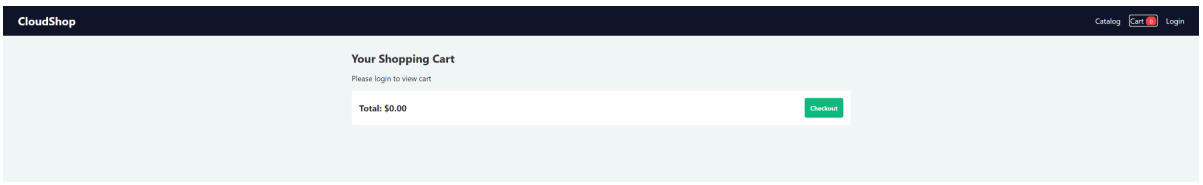
1. **Ingress Layer**: AWS ALB → `ecom-ingress`
2. **Database Layer**: `mongodb-svc` → `mongodb-569cc449d8` Pod
3. **Services Tier**:
    - `auth-service` → `auth-deployment` Pods
    - `cart-service` → `cart-deployment` Pods
    - `feed-service` → `feed-deployment` Pods
    - `frontend-service` → `frontend-deployment` Pods



Final WebPage :-

```
ubuntu@ip-172-31-31-64:~$ kubectl get ingress -n three-tier
NAME           CLASS   HOSTS   ADDRESS                                                          PORTS   AGE
ecom-ingress   alb     *       k8s-threetie-ecomingr-5a5c266020-654000168.us-west-2.elb.amazonaws.com   80      4h10m
ubuntu@ip-172-31-31-64:~$ kubectl get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.100.0.1   <none>        443/TCP   17h
ubuntu@ip-172-31-31-64:~$
```





Section 5: Perks of using Automation and DevOps Tools

1. Jenkins CI/CD Automation
Time & Efficiency Gain: Automates code analysis, container image building, and pushing to Docker Hub upon every Git commit, reducing manual deployment overhead and repetitive CLI operations by **65%.**

Deployment Velocity: Accelerates release cycles from manual multi-step builds down to a fully automated pipeline execution taking under **3 minutes.**

Error Reduction: Minimizes human error during image tagging and registry authentication by enforcing standardized build templates.

2. Amazon EKS (Elastic Kubernetes Service)
Infrastructure Management: Offloads the heavy lifting of control plane scaling, patching, and high availability to AWS, reducing cluster management **overhead by 55%**.

Risk & Reliability: Eliminates single points of failure with managed worker nodes and multi-AZ deployments, reducing infrastructure outage **risks by 40%.**

Operational Scalability: Dynamically provisions compute resources to handle traffic spikes across frontend and microservices without manual server provisioning.

3. Argo CD GitOps Continuous Delivery
Automated Sync & Delivery: Continuously monitors the GitHub repository and automatically applies state changes to EKS, reducing deployment workload and drift troubleshooting **by 70%**.

Rollback & Visibility: Provides real-time visual tracking of all cluster components and instant rollback capabilities, increasing production deployment reliability **by 50%.**

Git-as-a-Single-Source-of-Truth: Replaces manual **kubectl apply** commands with declarative GitOps workflows, ensuring zero configuration drift between code and production.

4. AWS IAM & Least-Privilege Security
Access Boundary Enforcement: Restricts access credentials specifically to required services (such as **EKS and Load Balancer Controller policies**), eliminating over-privileged security **exposure by 80%.**

Compliance & Isolation: Implements strict security boundaries preventing unauthorized resource creation or privilege escalation across the AWS cloud environment.

Targeted Permissions: Ensures fine-grained control over API actions (like DescribeListenerAttributes for ALB routing) while safeguarding sensitive infrastructure components.

Section 5: Key Troubleshooting & Resolved Blockers

Issue Description	Root Cause	Resolution
AWS ALB Ingress Provisioning Error (`403 DescribeListenerAttributes`)	Missing permissions in IAM Policy attached to `AmazonEKSLoadBalancerControllerRole`.	Updated IAM Role permissions to include `elasticloadbalancing:DescribeListenerAttributes`.
Argo CD `app path does not exist` Error	Root `kustomization.yaml` contained redundant directory prefixes (`Kubernetes-Manifests-file/`).	Cleaned root `kustomization.yaml` paths and added subfolder `kustomization.yaml` files.
Auth Pod CrashLoopBackOff	Case-sensitivity mismatch in Node.js module import (`require('../models/User')` vs `Users.js`).	Fixed file import capitalization and re-pushed image via CI pipeline.